CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY

(CBIT)

PROJECT PROPOSAL

LabSubmit

Secure Laboratory Examination, Code Execution & Evaluation Platform

A proposal for institutional adoption as the department's programming lab submission, examination, execution and evaluation platform

Submitted by: V Pranav

Roll Number: 160125749063

Branch: Computer Science & Engineering — IoT and Cybersecurity including Blockchain Technology

Year: 2nd Year

Institution: Chaitanya Bharathi Institute of Technology (CBIT)

Submitted to: College Administration, CBIT

Date: 16 August 2026

1. Executive Summary

This proposal introduces LabSubmit, a secure digital laboratory examination and assessment platform built for CBIT. It covers roll-number-based student registration, lab and examination authoring, an in-browser multi-language IDE with live server-side code execution, structured evaluation and grading, and Secure Exam Mode — browser-level Exam Integrity Monitoring backed by a server-authoritative exam session.

LabSubmit is more than an online examination website. It replaces the current manual, spreadsheet- and file-sharing-driven process of assigning lab tasks, collecting submissions and recording marks with a single role-based system used by Administrators, Faculty and Students — and it turns a lab examination into a controlled, auditable session in which the timer, the attempt state, authorisation and the submission are held by the server rather than by the student's browser.

The platform has already been built and is functional end to end. This document sets out the problem it solves, the existing system it replaces, its features and architecture, how a semester would run on it at CBIT, what Secure Exam Mode does and — equally importantly — what it does not do, and the case for adopting it as the department's official lab examination platform.

2. Introduction & the Existing System

Programming laboratory courses at CBIT require faculty to assign coding tasks, verify that submitted code actually compiles and runs correctly, and record evaluation marks for every student across every lab session. An increasing share of those sessions are graded examinations rather than practice exercises, which raises a second requirement the present arrangement does not meet at all: conducting the session under conditions the department can defend if a result is later questioned.

The existing system — the arrangement LabSubmit would replace — is not a software system at all. In practice it consists of:

- **Distribution by hand** — problem statements read out, written on a board, or emailed to a section before the session begins.
- **Collection by transfer** — completed files returned on pen drives, over email, or copied to a shared folder at the end of the period.
- **Verification on the lecturer's machine** — each submission opened, compiled and run manually to check that it works.
- **Recording in a spreadsheet** — marks entered by hand, held locally, and circulated later.
- **Integrity by invigilation alone** — the only control over what a student does on the machine in front of them is a person walking the room.

The arrangement does not scale across sections, years and departments; it produces no audit trail; and during a timed examination it offers no control over the browser, no record of suspicious activity, and no reliable enforcement of the time limit.

LabSubmit was designed and built to close both sets of gaps using a modern web stack, structured specifically around how CBIT already identifies and organises students — by 12-digit roll number, branch, year and section — and is proposed here for evaluation as the department's laboratory examination platform.

3. Problem Statement

The current arrangement leaves two distinct classes of gap. The first is operational and concerns how work is issued, collected and marked. The second concerns the integrity of a timed examination, and is the gap this revision of the proposal addresses directly.

Operational gaps

- **Fragmented submission process** — no centralised system for issuing lab tasks or collecting submissions, relying instead on email or pen-drive handoffs.
- **No registration control** — no enforcement of authorised roll-number ranges during registration, leaving room for unauthorised or duplicate accounts.
- **No built-in code execution** — faculty must manually compile and run each submission on their own machine to verify correctness.
- **No audit trail** — no consistent record of submission timestamps, evaluation status or standardised feedback across sections.
- **Manual grading** — marks are recorded by hand in spreadsheets, prone to error, with no real-time visibility for students.

Examination-integrity gaps

- **Tab switching** — a student can move to another browser tab or window during an examination, and nothing detects or records that it happened.
- **External resource access** — reference material, messaging applications and AI assistants are available on the same machine, one keystroke away from the examination.
- **Copy and paste** — a complete solution can be pasted in from a file, a chat window or a neighbouring machine in a single action.
- **Browser manipulation** — developer tools, the context menu and drag-and-drop transfer provide routes around whatever the page intends to allow.
- **Unreliable client-side timers** — a countdown held in the student's own browser can be paused, edited or reset by reloading the page, at which point the time limit means nothing.
- **Weak exam-session integrity** — nothing distinguishes a legitimate attempt from a second window, a stale session, or a submission replayed after the window has closed.
- **No evidence when suspicious behaviour occurs** — when something looks wrong, there is no record of what happened, when it happened or how often.
- **Difficult review for faculty** — without a per-student record, a lecturer reviewing a suspicious attempt has nothing to examine and no basis on which to act fairly.

One boundary should be stated at the outset, because the rest of this proposal depends on it being understood. No software platform can eliminate cheating that happens in the physical world. A second computer, a mobile phone, a printed sheet or another person in the room lies outside the reach of any web application. What a platform can do is close the routes that run through the browser, make the exam session itself impossible to manipulate from the client, and leave a timestamped record whenever a browser-level route is attempted — so that invigilation is supported by evidence rather than left to rely on memory.

4. Objectives

1. Centralise lab task creation, examination delivery, submission and grading in one role-based platform.
2. Enforce controlled student registration via Admin-authorised roll-number ranges.
3. Provide an in-browser code editor with real multi-language compilation on the server, removing the need for manual local verification.
4. Run every examination as a server-authoritative exam session, so that the timer, the attempt state, authorisation and the submission are decided by the backend and cannot be altered from the browser.
5. Reduce the browser-level opportunities for academic dishonesty during a timed examination through Secure Exam Mode — fullscreen, focus, visibility, clipboard and context-menu monitoring and restriction.
6. Record every detected integrity event with its type and a server-side timestamp, and present the sequence to faculty as reviewable evidence rather than as an automated verdict.

7. Give faculty a structured evaluation workflow — marks, remarks, status, the integrity log for each attempt, and the ability to reopen a workspace for correction.
8. Give students a stable, transparent examination experience: a reliable timer, clear warnings and a submission process that cannot silently fail.
9. Give administrators system-wide visibility into submissions, grading completion and examination integrity across faculty.
10. Be honest, in the platform and in its documentation, about the boundary between browser-level integrity monitoring and operating-system-level lockdown.

5. Proposed System Overview

LabSubmit is a centralised laboratory examination platform with three access levels, each with a dedicated dashboard:

Role	Login Identity	Primary Responsibility
System Administrator	admin@cbit.in	Faculty accounts, roll-number range allocation, institution-wide reporting
Faculty Lecturer	username@cbit.in	Section & lab management, submission review, grading
Student	12-digit CBIT roll number	Registration, coding, submission, grade viewing

Students register using their 12-digit CBIT roll number against Admin-approved ranges, are assigned to lecturer-managed sections, and complete lab work and examinations in a browser-based IDE. When an examination begins, the attempt is created and timed by the server, Secure Exam Mode activates in the browser, and the workspace locks automatically on submission — whether the student submits, the server deadline passes, or a faculty-set integrity threshold is crossed. Faculty author and grade examinations within their own sections, with the integrity log for each attempt beside the submission; Administrators manage faculty accounts, roll-number ranges and institution-wide reporting.

6. Innovation

What distinguishes LabSubmit from a general-purpose learning management system, or from a quiz tool with a timer, is that the examination itself is a first-class object with its own state, its own security model and its own audit record.

- **Secure Exam Mode** — a dedicated examination mode that monitors fullscreen state, window focus and tab visibility, intercepts developer-tool shortcuts, and restricts clipboard, context-menu and drag-and-drop actions for the duration of an attempt.
- **Server-authoritative exam sessions** — the deadline, the attempt state, authorisation and the submission are computed and enforced by the backend on every protected request. The browser displays the exam; it does not decide anything about it.
- **The integrity event timeline** — each detected event is stored with its type and a server-side timestamp against the student's attempt, producing a reviewable sequence rather than a single flag.
- **A faculty review dashboard** — per-exam and per-section violation logs, per-student counts, and the event list shown beside each submission at the moment a mark is being decided.
- **Automated coding assessment** — submissions are compiled and executed on the server within the platform, so correctness is verified where the marks are recorded, not on a lecturer's laptop afterwards.

- **Randomised assessments** — proposed as a future anti-collusion capability: variant problem statements and test data distributed across a section so that neighbouring students do not receive identical work.
- **A LabSubmit Secure Exam Client** — proposed as future work, providing operating-system-level lockdown that a web application cannot. It is described in Section 18 and is not part of the platform today.

The last two items are explicitly future work. They are listed here because the innovation the platform claims is a coherent path from browser-level integrity monitoring towards full examination lockdown — not a claim to have arrived at the end of it.

7. Key Features

7.1 Administrator

- **Faculty management** — create, edit, and delete lecturer accounts (username@cbit.in), set student capacity limits, and reset passwords.
- **Roll-number allocation** — define authorized roll-number ranges (e.g., 160125000001–160125000120) and assign them to specific lecturers.
- **Institution-wide student directory** — view all registered students across departments, automatically sorted by roll number.
- **System metrics & reporting** — view total submissions, grading completion rates, and performance statistics across faculty.

7.2 Faculty Lecturer

- **Section management** — create, rename, edit, and delete sections for 1st–4th year students (e.g., CSE-1, CIC-1).
- **Lab assessment authoring** — write problem statements, set due dates, and specify input/output constraints for lab tasks.
- **Submission viewer & code runner** — inspect multi-file student submissions, execute code live, and download source files.
- **Evaluation & grading** — award marks (0–100), write feedback remarks, set status (Approved / Rejected / Needs Correction), reopen workspaces for correction, and toggle instant grade publishing.

7.3 Student

- **Controlled self-registration** — registration is validated against a valid 12-digit roll number within an Admin-authorized range, preventing duplicate or unauthorized sign-ups.
- **Monaco online IDE** — a tabbed, multi-file online IDE with syntax highlighting for C, C++, Java, Python, and JavaScript.
- **Secure Exam Mode** — during an examination, fullscreen, window focus and tab visibility are monitored, and copy, paste, cut, right-click and drag-and-drop are restricted according to the settings the faculty chose for that exam.
- **Server-timed session and single-submission locking** — the countdown is issued by the server; the workspace becomes read-only immediately after submission unless reopened by the lecturer.
- **Real-time grade view** — published marks, evaluation status, and faculty remarks are visible immediately.

7.4 Secure Exam Mode

Secure Exam Mode is the examination mode of the platform, enabled per exam by the faculty who authored it. It is described in full in Section 8; in summary it provides:

- **Exam scheduling and duration** — a start time, an end time and a per-student duration, with the effective deadline derived by the server.

- **Fullscreen enforcement** — fullscreen is requested when the attempt begins; exits are detected, counted, and gated behind a blocking prompt to re-enter.
- **Focus and visibility monitoring** — loss of window focus and hiding of the tab are detected and recorded.
- **Clipboard and context-menu restriction** — copy, cut, paste, drag-and-drop, text selection and middle-click paste are blocked in the page, with per-exam overrides.
- **Developer-tool shortcut interception** — the common shortcuts are intercepted and recorded.
- **Server-authoritative timing and submission** — the deadline and the submission are owned by the backend, and an expired attempt is finalised by the server whether or not the browser reports back.
- **Integrity event logging and faculty review** — every detected event is timestamped, deduplicated and stored for review.
- **Threshold auto-submission** — crossing a faculty-set fullscreen-exit threshold submits the attempt automatically and locks the workspace.

8. Secure Exam Mode & Exam Integrity Monitoring

This section describes the platform's examination security model in full, including its limits. It is written so that the boundary between what LabSubmit does and what it does not do is unambiguous.

8.1 Two levels of exam security

Examination security in a laboratory setting operates at two levels, and the difference between them determines what any platform can honestly claim.

- **Browser-Level Exam Integrity** — what LabSubmit provides today. A web application can monitor and restrict behaviour inside the browser: fullscreen state, window focus, tab visibility, clipboard operations, the context menu, drag-and-drop and developer-tool shortcuts. It can record each occurrence with a timestamp and present the record to faculty.
- **OS-Level Secure Lockdown** — what LabSubmit does not provide today. Restricting the operating system itself — preventing other applications from running, controlling network access, restricting screen capture — requires a dedicated LabSubmit Secure Exam Client or Windows kiosk environment installed on the examination machine. This is proposed as future work in Section 18.

Every statement in this proposal about examination security should be read against that distinction. Where this document says LabSubmit monitors or restricts something, it means at the browser level.

8.2 The server-authoritative exam session

The single most important design decision in Secure Exam Mode is that nothing which decides the outcome of an examination is kept anywhere the student can reach. The browser displays the exam; the server owns it.

- **The timer** — the effective deadline is the earlier of the exam's scheduled end time and the student's own start time plus the permitted duration. It is computed by the server, re-derived on every protected request, and sent to the browser only for display.
- **The attempt** — an attempt is created server-side when the student explicitly starts the exam, and one attempt exists per student per exam.
- **Authorisation** — every protected endpoint independently re-verifies the caller's role, ownership of the workspace, the exam window and the deadline before acting. This includes the WebSocket channel that compiles and runs code.
- **The submission** — manual submission, expiry of the server deadline and threshold auto-submission all pass through one idempotent server-side routine, so the three paths cannot diverge or produce duplicate records.

The practical consequence is that a student cannot legitimately extend an examination by altering the countdown displayed in the browser, editing JavaScript state, clearing local storage, changing the computer's system clock, or replaying a network request. An attempt that is past its deadline is finalised by the server on the next request that touches it, whether or not the browser ever reports back.

8.3 Integrity events and severity

Each detected event is debounced in the browser, deduplicated on the server, and stored with its type, its context and a server-side timestamp against the student's attempt. The table below sets out the event taxonomy and the severity classification, and states plainly which events the platform records today and which are proposed as part of the next phase of work.

Severity	Integrity event	Status in the platform today
LOW	Copy attempt	Blocked in the browser; event logging proposed
LOW	Cut attempt	Blocked in the browser; event logging proposed
LOW	Context-menu attempt	Blocked in the browser; event logging proposed
LOW	Drag-and-drop attempt	Blocked in the browser; event logging proposed
MEDIUM	Fullscreen exit	Recorded
MEDIUM	Tab hidden or window minimised	Recorded
MEDIUM	Window blur (focus lost)	Recorded
MEDIUM	Developer-tools shortcut	Recorded
MEDIUM	Paste attempt	Blocked in the browser; event logging proposed
HIGH	Repeated fullscreen exit reaching the auto-submit threshold	Recorded
HIGH	Repeated focus loss within a short window	Proposed
HIGH	Duplicate exam session	Proposed
CRITICAL	Unauthorised exam access	Refused by the backend; event logging proposed
CRITICAL	Unauthorised submission	Refused by the backend; event logging proposed
CRITICAL	Backend authorisation violation	Refused by the backend; event logging proposed

Integrity events are signals for review, not automatic proof of academic misconduct. The platform never alters a mark on the strength of an event. The only automatic consequence anywhere in the system is submission of the attempt when a faculty-set fullscreen-exit threshold is crossed, and that threshold is set by the lecturer who authored the exam.

8.4 Faculty integrity review

Detection is only useful if the person deciding the mark can see what was detected. Faculty review integrity events in three places:

- **The violation log** — every event for the exams a lecturer owns, filterable by year and section, showing the student, roll number, exam, event type, detail and timestamp.

- **Per-student counts** — a running total per student per exam, shown alongside the submission list so that an unusual attempt is visible without opening it.
- **The submission inspector** — the events for that student's attempt are shown beside the code at the moment the mark is being assigned.

The presentation is deliberately evidential. LabSubmit shows a sequence of timestamped occurrences; it does not display a verdict, a score of suspicion or a recommendation. A student who left fullscreen once because a system notification stole focus looks different from a student who left it nine times, and the platform's job is to make that difference visible to a human rather than to decide it.

8.5 What Secure Exam Mode does not do

A web application cannot reliably control the Windows operating system. Secure Exam Mode therefore makes none of the following claims:

- **It does not lock the student's computer.** The machine remains fully under the student's control outside the browser.
- **It does not prevent Alt + Tab or Windows + Tab.** Switching away is detected and recorded; it is not prevented.
- **It does not block screenshots or screen recording.** Neither is visible to a web page.
- **It does not prevent the use of a second device or a mobile phone.** Nothing that happens off the examination machine is within reach.
- **It does not restrict Task Manager, USB devices or other applications.** These are operating-system concerns.
- **It does not use a camera, microphone or screen capture.** No biometric or visual surveillance of any kind is performed.

What it does do is close the routes that run through the browser itself, make the exam session impossible to manipulate from the client, detect when a student leaves the examination window, and record each occurrence for review. It supplements invigilation; it does not replace it. Stronger restrictions require the Secure Exam Client described in Section 18.

9. System Architecture & Technology Stack

LabSubmit is built as a modern, self-hostable web application using open-source technologies throughout, with no licensing cost to the institution. Roles enter through a single web application; every action that affects an examination passes through the Secure Exam Session boundary, where the Exam Integrity Engine and the Exam Engine sit behind a backend API that re-authorises each request before it reaches the database.

Layer	Technology	Purpose
Frontend	Next.js 14 (App Router, TypeScript), Tailwind CSS	User interface for Admin, Faculty, and Student portals
Code Editor	Monaco Editor	In-browser multi-file IDE for C, C++, Java, Python, and JavaScript
Backend / API	Next.js API Routes, custom Node.js server (ws, node-pty)	REST endpoints and real-time terminal/process handling
Database / ORM	Prisma ORM — SQLite (dev) / PostgreSQL (production)	Stores users, sections, labs, submissions, grades
Authentication	JWT (jsonwebtoken), bcryptjs	Role-based session tokens and password hashing

Execution Engine	Native GCC, G++, javac/java, Node.js invocation	Compiles and runs C, C++, Java, Python, and JavaScript submissions
Deployment	Vercel (web + API), Railway (execution engine + PostgreSQL), Docker	Live cloud deployment; the same Docker image also runs on a college server
Exam Integrity	ExamGuard, AntiCheatWrapper, ExamViolation event store	Secure Exam Mode monitoring, restriction and integrity event recording
Exam Session Control	Server-side exam timing, authorisation and submission services	Server-authoritative deadline, attempt state, one-submission locking

10. How LabSubmit Is Used at CBIT

The platform is designed to mirror how a lab course actually runs at CBIT across a semester, without departing from the existing faculty, section, and roll-number structure already used administratively:

11. The Administrator sets up the term — creating faculty accounts for lab instructors and defining the roll-number range for each incoming section or branch (for example, CSE – IoT & Cybersecurity including Blockchain Technology, 2nd year), and assigning that range to the relevant lecturer.
12. Students self-register with their CBIT roll number and a password. Registration is validated live against the range assigned by the Admin, which prevents unauthorized or out-of-range sign-ups.
13. The Lecturer creates sections (for example, “CSE - 2”) and authors lab assignments — problem statements, input/output constraints, and due dates.
14. During the scheduled examination, students log in, open the assigned exam and start it explicitly. The server creates the attempt and begins the countdown, fullscreen is requested, and Secure Exam Mode monitors the session while the student writes and tests code in the in-browser IDE, compiling and running it live against the platform's execution engine.
15. Students submit their work. The workspace locks automatically — on the student's own submission, at the server-side deadline, or on crossing the fullscreen-exit threshold the lecturer set — so that it cannot be edited afterwards.
16. The Lecturer reviews submissions in the grading panel with the integrity log for that attempt beside the code, re-runs the submission where needed, assigns marks and remarks, sets an evaluation status and publishes grades — or reopens the workspace if a student needs to correct and resubmit.
17. Students see their marks, status, and feedback in real time on their dashboard, without waiting for manually compiled results.
18. At any point, the Administrator can pull system-wide metrics — total submissions and grading completion rate across faculty — for departmental reporting.

In practice, this means a lab examination that currently depends on pen drives, email and a lecturer's personal machine to compile code is instead run entirely through one platform that already reflects CBIT's roll-number and section structure — from registration, through a monitored and server-timed examination, to final grade publication with an audit trail behind it.

11. Implementation & Deployment Plan

- **Phase 1** — Deploy for one section (e.g., CSE – IoT & CSBCT, 2nd year) for a single lab course, on the existing live deployment (Section 14) — requiring no college server, no procurement, and no setup cost.
- **Phase 2** — Admin account setup, faculty onboarding, and roll-number range configuration for the pilot batch.

- **Phase 3** — Trial: run 2–3 lab sessions on the platform alongside the existing process, and collect faculty and student feedback.
- **Phase 4** — Rollout: expand to additional sections and years based on pilot results, moving to the paid hosting tier or to college infrastructure as set out in Section 15.3.
- **Ongoing** — maintenance, support, and long-term continuity are addressed in Section 19. Lab computers require no compilers or software installation of any kind — everything runs in the browser.

12. Expected Impact & Benefits

For students

- **A structured examination experience** — a clear pre-exam briefing, an explicit start, and rules that are stated before the attempt begins rather than discovered during it.
- **A reliable timer** — the countdown comes from the server, so a slow machine, a reloaded page or a background tab does not cost a student time.
- **Transparent warnings** — restricted actions produce an immediate, readable message rather than silent failure, and the student can see their own fullscreen-exit count against the threshold.
- **A stable submission process** — one submission path, an explicit confirmation, and no dependence on a file transfer succeeding at the end of the period.

For faculty

- **Centralised examination management** — authoring, scheduling, monitoring and grading in one panel, within their own sections.
- **Automatic execution where applicable** — submissions compile and run inside the platform, removing manual verification on a personal machine.
- **Integrity event visibility** — the events for an attempt are available at the moment the mark is being decided, not reconstructed afterwards.
- **Easier review of suspicious attempts** — a timestamped sequence per student replaces recollection and suspicion as the basis for a conversation with a student.
- **Standardised grading** — consistent marks, remarks and status categories across sections and faculty.

For the institution

- **Scalable digital lab examinations** — the same platform runs one pilot section or several departments without changing how students are identified administratively.
- **Improved auditability** — submission timestamps, evaluation records and integrity events are retained and exportable.
- **Centralised assessment infrastructure** — one system replaces email, pen drives and spreadsheets across every lab session.
- **Accreditation reporting** — real-time visibility into lab performance and grading completion, useful for outcome-based NBA / NAAC reporting.
- **A foundation for secure kiosk-based examinations** — the server-authoritative session model is the prerequisite for the Secure Exam Client described in Section 18, so the work done now is not discarded later.
- **No change to existing structure** — built around CBIT's existing roll-number, branch and section structure.

13. Feasibility & Resource Requirements

- **Software** — already built, deployed, and functional — a complete Next.js/TypeScript codebase with a Prisma database schema, Docker configuration, and a live public deployment.

- **Hosting** — already live on cloud hosting (see Section 14). The same Docker image runs unchanged on a college server or VM with GCC, G++, JDK 17+, Python 3, and Node.js installed — see Section 15.3 for the cost comparison.
- **Database** — PostgreSQL in the current deployment, which scales from a single pilot section to multi-department use without further change.
- **Cost** — built entirely on open-source frameworks (Next.js, Prisma, Monaco Editor) — no licensing fees. Infrastructure costs are set out in full in Section 15.
- **Training** — role-based dashboards are self-explanatory; minimal training is needed for Admin, Faculty, or Students to get started.

14. Live Deployment & Access

LabSubmit is not a prototype awaiting funding to be built — it is deployed and reachable now. The committee can open it, log in, and use it before deciding anything.

Item	Detail
Platform URL	https://labsubmit.vercel.app
Application hosting	Vercel — serves the web interface and all REST API endpoints
Execution engine hosting	Railway — runs the compiler container (GCC, G++, OpenJDK 17, Node.js, Python 3) and the live terminal connection
Database	Managed PostgreSQL (Railway), with all data encrypted in transit
Encryption	HTTPS/TLS enforced on every request; certificates auto-renewed
Source code	github.com/vudigapranav/LabSubmit — available for institutional review
Demonstration access	Reviewer credentials supplied separately to the evaluating committee

The two hosts are split by necessity, not preference: compiling and running student code requires a persistent process with real compilers attached, which the application host does not provide. Students and faculty see one website; the split is invisible to them.

The current deployment runs on free and entry-tier plans, at no cost to the institution. It is fully functional for demonstration and for a supervised pilot. Section 15 sets out what it would cost to run at department scale, and Section 17 is candid about what the free tier does not guarantee.

15. Budget & Cost Analysis

LabSubmit carries no licence fee and no development cost — the platform is already built and is contributed to the institution. The expenditure sought is infrastructure to run it, plus a student honorarium at institute norms for the twelve-month hardening and pilot work. This section sets out those costs in full, including the ones that are zero, so the committee can see there is nothing hidden.

All figures in Indian Rupees, inclusive of 18% GST where applicable. Foreign-currency hosting converted at ₹95 per USD (August 2026). Cloud pricing is usage-based and may vary by ±10%. No cost of any kind is passed on to students.

15.1 Phase 1 — Pilot (one section, one semester)

The pilot is deliberately costed to remove any financial barrier to trying the platform. It runs on free and entry-tier hosting on the Vercel-supplied web address, so no procurement, purchase order, or domain request is needed to begin.

Item	Basis	Cost (₹)
Web address	Vercel-issued subdomain (labsubmit.vercel.app)	0
Application hosting	Vercel free tier	0
Execution engine	Railway entry plan, ~₹475/month × 6 months	2,850
Database	PostgreSQL, included in the above	0
SSL certificate	Let's Encrypt, auto-provisioned	0
Software licences	All components open-source	0
Development & setup	Contributed by the student authors	0
Total — full pilot semester		**2,850**

The pilot can also be run entirely on trial credit at ₹0, accepting the idle-restart delay noted in Section 17.

15.2 Phase 2 — Department-wide deployment (annual infrastructure)

Once the platform carries graded assessments for a full department (approximately 300 students), it needs paid plans — for guaranteed uptime, for a container that does not sleep between lab sessions, and for licence terms that permit institutional use. The heads below match, item for item, the budget submitted with the Software Development Project application.

Item	Basis	Annual cost (₹)
Domain name	labsubmit.cbit.ac.in — issued by college IT as a subdomain of the existing institutional domain	0
(alternative)	An independent .in domain, if a college subdomain is not preferred	1,100
SSL / HTTPS certificate	Let's Encrypt, renewed automatically by the host	0
Application hosting	Vercel Pro, 1 seat — ₹1,900/month × 6 active academic months	11,400
Execution engine hosting	Always-on 1 vCPU / 1 GB container — ~₹1,900/month × 12	22,800
Database (PostgreSQL)	Managed instance, included in the container allocation above	0
AI / LLM API services	Similarity explanation, feedback drafting and test-case generation — metered credits, ~₹750/month × 12	9,000
Backups, monitoring, load testing	Nightly snapshots with 30-day retention, uptime alerting, and staging resources for concurrency trials	6,000
Software licences	Next.js, Prisma, PostgreSQL, GCC, OpenJDK — all open-source	0

Development & maintenance	Contributed; no vendor contract required	0
Domain, SSL and contingency	Compute overage during peak examination weeks	3,800
Total — infrastructure per year		**53,000**

Added to this, the Software Development Project application seeks a student honorarium of ₹36,000 (2 student developers × ₹2,000/month × 9 active project months, as per institute norms) for the production-hardening, integrity-module and load-testing work described in that application.

Head	Amount (₹)
Infrastructure (annual, as above)	53,000
Manpower — student honorarium	36,000
Total requested — twelve-month project	**89,000**

Under ₹4,500 a month for the entire department, for a platform that replaces manual code compilation, pen-drive submissions, and spreadsheet grading across every lab session.

15.3 Three-year projection

Two paths are presented honestly, because the cheaper one is not the one that gets the platform running fastest. Scenario A keeps everything on commercial cloud hosting. Scenario B pilots on the cloud and then moves onto college infrastructure, which the platform already supports — it ships as a Docker container and runs unchanged on a campus server.

Scenario A — Cloud hosted	Year 1	Year 2	Year 3	3-year total
Scope	Grant year, 1 dept	Full department	2–3 departments	—
One-time cost	0	0	0	0
Recurring cost	89,000	53,000	53,000	1,95,000
Total	**89,000**	**53,000**	**53,000**	**1,95,000**

Scenario B — Cloud year one, then college-hosted	Year 1	Year 2	Year 3	3-year total
Scope	Cloud, grant year	Migrate to college VM	Steady state	—
One-time cost	0	0 – 75,000	0	0 – 75,000
Recurring cost	89,000	0	0	89,000
Total	**89,000**	**0 – 75,000**	**0**	**89,000 – 1,64,000**

Scenario B shows ₹0 one-time cost in Year 2 if CBIT already operates a virtual-machine host with spare capacity, which most institutional IT departments do; ₹75,000 covers a dedicated server if none is available.

Recommendation: begin with Scenario A and migrate to Scenario B. Cloud hosting gets the pilot running the same week it is approved, with no procurement and no demand on college IT. Once the platform has proven itself, moving it onto campus infrastructure removes the recurring cost almost entirely, keeps student data physically within the institution, and pays for itself against Scenario A within the second year.

15.4 What the budget deliberately does not include

Several items that normally appear in a software budget are genuinely absent here, and are listed so their absence is understood as accurate rather than overlooked:

- **Licence fees — ₹0.** Every component is open-source under MIT, Apache 2.0, or GPL. There are no per-student seats, no annual renewals, and no vendor lock-in.
- **Development cost — ₹0.** The platform is complete and functional, built as a student project and offered to the institution.
- **Training cost — ₹0.** The three dashboards are self-explanatory; a single one-hour walkthrough per faculty group is sufficient, and can be delivered by the authors.
- **Migration cost — ₹0.** There is no legacy system to migrate from — the current process is email, pen drives, and spreadsheets.
- **Hardware for students — ₹0.** LabSubmit runs entirely in a web browser. Existing lab computers need no compilers, no editors, and no software installation of any kind.
- **Any charge to students — ₹0.** No part of this budget is recovered from students in any form.

That last point about hardware is worth weighing on its own: because compilation happens on the server, lab machines no longer need GCC, JDK, or an IDE installed and kept in sync. That is a real reduction in the ongoing configuration workload carried by college IT, and it is not counted as a saving anywhere in the tables above.

16. Data Security, Privacy & Ownership

The platform holds student names, roll numbers, submitted code, and marks. How that data is protected, and who owns it, is set out here.

- **Password storage** — passwords are hashed with bcrypt and are never stored or transmitted in readable form. Nobody, including the administrator, can retrieve a student's password; it can only be reset.
- **Session security** — access is governed by signed JWT tokens carrying the user's role. Every API endpoint re-verifies the role before returning data, so a student cannot reach faculty or administrative functions by manipulating a request.
- **Encryption in transit** — all traffic, including the live terminal connection, runs over TLS. No credential or submission travels unencrypted.
- **Execution authorisation** — before any code is compiled, the server independently verifies the student's identity, that they own the workspace, that the examination window is open and that the deadline has not passed. The browser is treated as untrusted throughout.
- **Server-authoritative exam state** — the examination deadline, the attempt state and the submission are computed and enforced by the backend. Values sent by the browser are never accepted as authoritative, so exam state cannot be altered from the client.
- **Integrity event retention** — integrity events are written with a server-side timestamp and retained beyond submission, so a decision about a student can be revisited with the full sequence available.
- **Monitoring scope and student privacy** — Secure Exam Mode captures interaction metadata only: which browser events occurred and when. No camera, microphone, screen capture, keystroke content or personal file access is used anywhere in the platform.
- **Data ownership** — all data belongs to CBIT. There is no third-party analytics, no advertising, no data sharing, and no external processing beyond the hosting providers themselves.
- **Data residency** — under the college-hosted option in Section 15.3, all student data remains on campus infrastructure and never leaves the institution.
- **Backups & retention** — nightly database snapshots with 30-day retention; retention beyond that follows whatever policy the college sets for academic records.

- **Portability** — data lives in a standard PostgreSQL database and can be exported in full at any time. The institution is never locked in.

17. Risks, Limitations & Mitigation

The platform is presented as it actually is, including where it is not yet ready. A pilot is proposed precisely so these limits are tested at small scale rather than discovered during a graded examination.

Risk / limitation	Impact	Mitigation
No isolation between running submissions	Student code executes as an ordinary process in a shared container with no per-student CPU or memory limit. One infinite loop can slow the session for others.	Acceptable under supervision at pilot scale. Per-execution resource limits and container isolation are the first priority before department-wide rollout.
Hosting outage during a live lab session	Students cannot submit within the assessment window.	Uptime monitoring with alerts; the existing offline procedure is retained as a fallback throughout the pilot.
Free-tier hosting has no uptime guarantee	Idle services restart slowly; the licence terms of free plans do not cover institutional use.	Paid plans are budgeted in Section 15.2 before the platform carries any graded assessment.
Secure Exam Mode is browser-level, not OS-level	Fullscreen, focus, visibility, clipboard and context-menu behaviour are monitored and restricted inside the browser, but the platform cannot control the operating system. Alt + Tab, Windows + Tab, Task Manager, screenshots, screen recording, USB devices, a second computer and a mobile phone all remain outside its reach.	The platform supplements invigilation, it does not replace it. Every browser-level event is recorded with a timestamp for faculty review, and fullscreen-exit counts are surfaced against a faculty-set threshold. A dedicated LabSubmit Secure Exam Client providing OS-level lockdown is proposed as future work in Section 18.
Single-maintainer dependency	The platform currently has one developer.	Source code, schema, and deployment documentation are handed to the department — see Section 19.
Concurrent load at department scale is untested	Behaviour with 300 simultaneous compilations is not yet measured.	The pilot establishes real load figures before any wider commitment is made.

18. Future Scope

The work below is not implemented. It is set out here so that the scope of what exists today is not misread, and so that the committee can see that the platform has a coherent direction rather than a fixed ceiling.

18.1 LabSubmit Secure Exam Client

A dedicated desktop client, installed on examination machines, would provide the operating-system-level restrictions that a web application cannot. Proposed capabilities:

- **Windows kiosk mode** for the duration of an examination session.
- **Stronger application restrictions** — controlling which applications may launch while an exam is in progress.
- **Controlled network access** — restricting reachable destinations during an exam.
- **Stronger OS-level exam lockdown** — closing the routes that browser-level monitoring can only observe.

- **A controlled lab environment** — a reproducible examination configuration rather than whatever state a machine happens to be in.
- **Centralised deployment** — installation and configuration managed across institutional labs from one place.

The client would continue to use the same server-authoritative exam session, the same integrity event store and the same faculty dashboard. Nothing built now would be discarded; the client would add a stronger enforcement layer beneath an unchanged model. It is not part of the twelve-month plan and no budget head in Section 15 depends on it.

18.2 Other planned work

- **Per-student execution sandboxes** — CPU, memory, process and time limits per execution, with an execution queue, replacing the present shared container.
- **A formal event severity model** — persisting the Low / Medium / High / Critical classification described in Section 8.3 and extending recording to clipboard, context-menu and duplicate-session events.
- **Randomised assessments** — variant problem statements and test data across a section, as an anti-collusion measure that reduces the value of copying in the first place.
- **Similarity detection** — structural comparison across a section, with AI-drafted explanations and remarks that faculty accept, edit or discard. No mark would be assigned automatically.
- **Batch tooling and exports** — reference solutions and test cases per exam, batch execution of a section, mark-sheet export and accreditation reports.

19. Maintenance, Support & Continuity

A fair question about any student-built system is what happens when the student graduates. It is answered directly here rather than left for the committee to raise.

- **During the pilot** — the authors maintain the platform, resolve issues, and support faculty at no cost to the institution.
- **Documentation handover** — the source code, database schema, deployment guide, and administrator manual are handed to the Department of CSE, so the platform can be operated and modified without the authors.
- **Built on standard, widely-taught technology** — Next.js, TypeScript, PostgreSQL, and Docker are mainstream tools. Any competent developer, including a final-year student under faculty supervision, can maintain the codebase.
- **Continuity as a student project** — the platform is well suited to being carried forward by successive project batches under faculty guidance, which both sustains it and gives students real production experience.
- **No vendor lock-in** — nothing proprietary is used. Should the college choose to stop using LabSubmit, the data exports cleanly and no contract has to be exited.
- **Institutional ownership** — the authors are willing to transfer the codebase to CBIT outright, so the platform remains the department's asset regardless of who maintains it.

20. Conclusion

LabSubmit is a working, deployable platform that addresses the operational gaps in how programming lab courses are currently run at CBIT — from roll-number-controlled registration through to live code execution and structured grading — and, with Secure Exam Mode, the integrity gaps that appear the moment a lab session becomes a graded examination. It does not merely conduct examinations online. It provides a controlled, auditable examination environment with server-authoritative exam sessions and browser-level

Exam Integrity Monitoring, together with a clear and honestly stated path towards operating-system-level lockdown through a future Secure Exam Client.

This proposal requests the college's consideration to trial LabSubmit for an upcoming lab course as a step towards adopting it as the standard platform for programming laboratory management and examination at CBIT.